AsmWebIR

LLM Decompilation by Forward Edits:
Webs, Holes, Pantograph Drafts, Shared Memory, and Many Agents

Consolidated draft — 2026-10-01 — supersedes Parts I–III

Summary. Decompilation is program synthesis whose specification is a binary and whose *hints* are the facts cheaply read from it. LLMs are good at guessing the parts the binary does not contain: names, types, intent, idiomatic structure. They are bad at staying consistent with the binary and with themselves. We give the LLM an engine instead of a text box. The engine keeps a store of **facts that only grows**. Facts from the binary are certain. LLM guesses are labelled **claims**. Contradictions are recorded with a machine-level witness, and undo is itself a fact. The program an LLM sees is a **view** computed from those facts. It edits a scratch **draft** of that view with a typed structure editor (Pantograph) and submits the draft as claims. **Webs** make one claim apply to every place that must agree with it. On top of this we describe a practical system: a retrieval memory over the edit history and the program's entities, and a multi-agent protocol in which agents recover functions in isolation and synchronise at function-completion barriers. Part A tours the idea on examples, Part B designs the system, and Part C collects every formal rule, each with a one-line explanation.

Contents

Part A — A guided tour . . . 3
1 The problem, in one paragraph . . . 3
2 The running example . . . 3
3 Step 0 — the engine alone . . . 3
4 Step 1 — the LLM makes a claim . . . 4
5 Step 2 — a hallucination . . . 5
6 Step 3 — signatures across an indirect call . . . 5
7 Step 4 — restructuring in a draft . . . 6
8 Step 5 — raising residual code . . . 6
9 Step 6 — undo, and what is learned . . . 6
10 Step 7 — two agents at once . . . 7
11 What the idealised system guarantees . . . 7
12 Where the ideas come from . . . 7
Part B — A realistic system . . . 8
13 Architecture . . . 8
14 Memory: retrieval over edits and entities . . . 8
14.1 What is indexed . . . 8
14.2 Rules for using retrieval safely . . . 9
14.3 Context assembly for an agent . . . 9
15 Agents: isolation, completion barriers, synchronisation . . . 9
15.1 Work units and scheduling . . . 9
15.2 Local and global claims . . . 10
15.3 The protocol . . . 10
15.4 Conflict policy . . . 10
15.5 Worked timeline on the running example . . . 11
16 The agent's tool interface . . . 11
17 Engineering notes . . . 11
18 Evaluation plan (sketch) . . . 12
Part C — The formal rules . . . 13
19 Syntax . . . 13

19.1 Types, kinds, terms 13
19.2 Knowledge 13
20 Kinds and layouts 13
21 Typing of terms 14
22 Webs and locality 15
23 The knowledge store: saturation 16
24 Views 17
25 Drafts: Pantograph editing 17
25.1 Propagation 17
25.2 Alteration (where a boundary stops) 18
25.3 Healing 18
25.4 Statement spans 18
26 Edits and submission 18
27 Multi-agent protocol 19
28 Memory (retrieval) rules 20
29 Properties we expect (conjectures) 21
30 Superseded rules 21
References 21

Part A — A guided tour

1 The problem, in one paragraph

Ask an LLM to decompile a function and you get plausible C. But the struct it invents may put an `int` where the binary reads a pointer. The caller, decompiled separately, may disagree about that struct. A parameter may silently disappear, a loop bound may be off by one, and a "small" type change may contradict ten functions the model never saw. The model is not tied to the binary, and it is not tied to its own earlier answers. The rest of this document is about building those two ties.

> **Thesis.** Decompilation = synthesis with hints. The binary B fixes *what* the program does, and the LLM proposes *how it was written*. Every proposal is recorded, checked against B where possible, and propagated to every place it constrains. Nothing is ever silently accepted or silently lost.

2 The running example

A small x86-64 (System V) program. Some parts are shown as assembly, the rest summarised.

```
; ---- sub_401000: sums a linked list -----------------------------------
401000: xor  eax, eax
401002: test rdi, rdi
401005: je   401012
401007: add  eax, dword ptr [rdi+0x8]     ; 4-byte int at offset 8
40100a: mov  rdi, qword ptr [rdi]         ; pointer at offset 0, same kind of object
40100d: test rdi, rdi
401010: jne  401007
401012: ret

; ---- sub_401020(head, v): malloc(16); [n+0]=head; [n+8]=v (dword); return n
; ---- sub_401060(p): [p+8] += 1 (dword); [p+0] = 0 (qword)   -- a counter, unrelated
; ---- main: h = 0; h = sub_401020(h,1); h = sub_401020(h,2); return sub_401000(h)
; ---- dispatcher @401200: rdx <- r12; rsi <- rbx; rdi <- r13; call [0x404000 + rax*8]
;        table entries: h_ping @401300 (reads rdi, rsi), h_data @401340 (reads rdi,
rsi, rdx)
; ---- sum16 @401400: for (rcx = 0; rcx != 16; rcx++) eax += dword [rdi + rcx*4]
; ---- iabs  @401500: branch-free cmov sequence
```

The tour runs in steps. After each step, green boxes say what the system can do at that point, and red boxes say what it cannot.

3 Step 0 — the engine alone

Before any LLM is involved, a mechanical lifter translates the binary into **AsmWebIR** and extracts **premises**: certain facts about the binary.

```
fn#c1 sub_401000(a0#w1: ptr#d1 ?T1) -> ?T2
{
  let v1#w2: ?T3 = 0;                                  @401000
  if (a0 != null) do {                                  @401002
    v1 = v1 + load[4,int](a0 + 8);                      @401007
    a0 = load[8,ptr](a0 + 0);                           @40100a
  } while (a0 != null);                                 @40100d
  return v1;                                            @401012
```

```
}
holes:  ?T1 :: [>=12, 8, agg]  :> { 0: ptr#d1 ?T1,  8: [4,4,int] }
        ?T3 :: [4,4,int]    ?T2 = ?T3
```

Reading it:

- **Holes** (`?T1`) are unknown types. Each has a **kind** (size, alignment, class) read from the instructions. `?T1` also has an **open layout**: at offset 0 there is a pointer to another `?T1`, and at offset 8 a 4-byte integer. The engine knows this without guessing.
- **Webs** (`#c…`, `#d…`, `#w…`) group things that must agree. `#d1` is every pointer that can flow to the same objects: `a0` here, `head`/`n` in `sub_401020`, `h` in `main`. `#d7`, the counter in `sub_401060`, is a **different** web, even though its layout looks almost the same.
- Every statement carries **provenance** (`@401007`), the instruction range it came from.
- Code the lifter cannot structure stays as **residual** assembly, `asm{401500..401512}`. It is typed, and exact.

The engine also derives facts across functions. `h_data` reads `rdx`, and all table entries flow to one indirect call, so they share code web `#c4`. Every function in `#c4` must therefore take at least three arguments. At the indirect call, the third argument is whatever `rdx` holds, i.e. the value copied from `r12`.

Can: a faithful, typed IR with every unknown explicit; kinds, offsets and open layouts for unknown structs; which pointers/functions must agree (webs); arity of indirect-call targets; the value passed in each argument register at each call.

Cannot: names; types beyond their kind (is `[4,4,int]` a `int32_t`, `uint32_t`, or an `enum`?); where one struct ends and another begins; what the program is *for*.

4 Step 1 — the LLM makes a claim

The LLM sees `sub_401000` with its holes, and claims:

```
a2 : declare struct Node { 0: next: ptr Node;  8: val: i32 } size 16
a3 : ?T1 = Node
a4 : name c1 "list_sum";   name w1 "node";   name w2 "total"
```

These go into the knowledge store as **claims**, labelled `a2`, `a3`, `a4`. The engine combines them with the premises. `Node` fits the open layout, so it derives that `load[4,int](a0 + 8)` *equals* `node->val`, and similarly for `next`. It derives this in every function of web `#d1`, not only the one the LLM was looking at:

```
fn#c1 list_sum(node: ptr Node) -> i32 {
  let total: i32 = 0;
  if (node != null) do {
    total = total + node->val;
    node = node->next;
  } while (node != null);
  return total;
}
fn#c2 list_push(head: ptr Node, v: i32) -> ptr Node {
  let n: ptr Node = malloc(16);
  n->next = head;
  n->val  = v;            // v: i32, solved by unification
```

```
  return n;
}
```

`sub_401060` (web `#d7`) is untouched. The raw forms are not erased: they stay in the store as equal alternatives, so a later "why is this `node->val`?" can show the load and the claims it rests on.

Can: apply one claim coherently to every function that must agree with it, and only to those; keep the evidence behind every displayed construct.

Cannot: decide whether the counter in `sub_401060` is secretly also a `Node`. No data flow connects them, so the engine keeps them apart. The LLM *may* claim they are the same web, but that claim is recorded as **uncorroborated**.

5 Step 2 — a hallucination

Suppose the LLM had instead claimed `8: val: i64`. The engine derives a contradiction, which is recorded as a **nogood**: a set of claims that cannot all be true.

```
nogood {a2, a3}: Node.val is [8,8,int]
    but 401007  add eax, dword ptr [rdi+0x8]  reads [4,4,int]
```

The view shows the raw load with a mark and the witness. The LLM retracts `a2` (with a reason) and claims the `i32` version. Nothing is deleted: the store keeps the wrong claim, its retraction and the nogood. Another agent that later proposes `val: i64` is answered from the record immediately.

Can: catch any claim that contradicts kinds, offsets, sizes, alignment, pointer-ness, arity, or the flow structure (webs); explain the contradiction with instruction addresses; remember it.

Cannot: catch plausible claims the binary does not constrain: `i32` vs `u32` when no signed or unsigned instruction touches the value; `val` vs `count` as a name; whether a `[8,8,int]` is a `size_t` or a `uint64_t` in the original source.

6 Step 3 — signatures across an indirect call

The engine has derived that web `#c4` has at least three parameters. Every member's view shows a third parameter hole, and the dispatcher shows `v12` (from `r12`) as the third argument. The LLM only adds what the binary cannot:

```
a5 : type(param 3 of c4) = u64        a6 : name(param 3 of c4) = "len"
```

`h_ping` now visibly takes `len` and ignores it. That is correct, not noise: every function that can be called through that table must share a calling convention.

Can: derive arities from register usage across whole webs; fill call arguments from the binary's dataflow instead of inventing them.

Cannot: recover variadic signatures exactly; distinguish "unused parameter" from "parameter the original source never had" when the calling convention does not force it; see through custom calling conventions unless the lifter models them.

7 Step 4 — restructuring in a draft

The LLM wants `while (node) { … }` instead of `if (node) do { … } while (node)`. There is no single typed edit for this, so it opens a **draft**: a scratch copy of the view, edited with Pantograph. It cuts the two conditions, unwraps the `do`, wraps a `while`, and merges the conditions. Midway, the draft is typed but **not faithful** to the binary: a condition is missing. That is fine, because drafts are not knowledge.

On **submit**, the engine compares the draft with the view. The only change is "this loop equals that loop". It already knows that equality from a loop-rotation rewrite, so the answer is *derived, certain*, and the view switches to the `while` form.

Can: let the LLM edit fluidly, through ill-formed and unfaithful intermediate states; accept any presentation change that the rewrite library proves equal.

Cannot: accept a presentation change outside the rewrite library without evidence. Such a change becomes a **raise** (Step 5), and may end up only *tested*, not proved.

8 Step 5 — raising residual code

`iabs` is a branch-free `cmov` sequence, left as residual assembly. The LLM proposes `(x < 0) ? -x : x`. The checker (symbolic execution or SMT over the region) either proves equivalence, or returns a counter-example. Here: `x = INT_MIN`, where the assembly gives `0x80000000` and the C expression has undefined behaviour. The LLM can revise to `(int)(x < 0 ? -(unsigned)x : x)`. If the checker times out, the raise is kept as **tested** (with the number of random inputs) and stays visible as an open obligation.

Can: replace machine code by source-level claims with a recorded status: proved, tested, or refuted with a counter-example.

Cannot: prove large regions with loops and memory in reasonable time; guarantee the absence of undefined behaviour in the output; reproduce byte-identical recompilation.

9 Step 6 — undo, and what is learned

The LLM wonders whether `total` should be 64-bit, and claims it. The engine derives a nogood at once (`add eax, …` is 32-bit). The LLM retracts the claim. The **view** is exactly as before, but the **store** now also knows "`total` is not `i64`". Undo never erases; it adds a retraction.

Can: undo anything; see exactly which view nodes depend on a claim before retracting it (the blast radius); keep every mistake as reusable knowledge.

Cannot: forget: the store only grows, and needs archiving over long sessions (Part B).

10 Step 7 — two agents at once

Agent A names web `#d1` "Node" while agent B, working on `list_push`, names it "ListItem". Both claims are facts. Adding facts is order-independent, so the agents do not interfere *mechanically*. The disagreement shows up as a naming conflict on the web, which one of them resolves by retracting. Had they disagreed about a *layout*, it would have been a nogood naming both claims.

Can: let many agents add claims concurrently; attribute every conflict to the claims (and agents) responsible.

Cannot: decide which of two unconstrained names is "right". That needs a policy (Part B, §sync).

11 What the idealised system guarantees

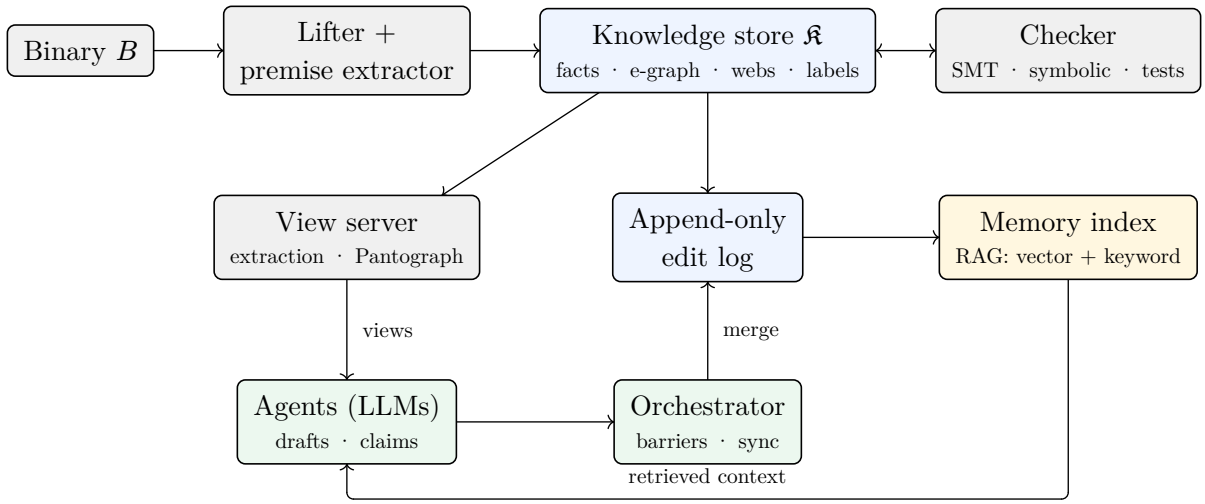
Guaranteed	Not guaranteed
Every construct shown is either derived from the binary by sound rewrites, or carries the labels of the claims it depends on	That claims about names, signedness, or intent are *true*
Claims that contradict kinds, layouts, arities or flow are detected, with a witness	That every wrong claim contradicts something (many are simply unconstrained)
Global consistency: things that must agree (a web) always agree in every view	Precision of webs: unification may merge more than necessary (e.g. through `void*` containers)
No information is lost; every decision is explainable after the fact	Bounded memory; the store must be archived
Raised code has a recorded status	Proof of every raise; absence of UB; recompilability

12 Where the ideas come from

- **Webs** [1]: producers and consumers that must agree on an interface, inferred by unification and carried in types. They give the "one claim, every place that must agree" behaviour of Step 1.
- **Pantograph** [2]: typed structure editing with one-hole contexts and type diffs, where errors are first-class boundaries. It is the draft editor of Step 4 and the view updater.
- **Hazel's marked lambda calculus** [3] and **Grove** [4]: every editor state is meaningful, errors live in holes with traced provenance, and collaborative edits commute. They give the marks of Step 2 and the concurrency of Step 7.
- **Tylr** [5]: temporarily breaking structure with obligations to repair it, as in the loop restructuring of Step 4.
- **E-graphs and egglog** [6], [7], and **Szalinski** [8]: rewrites add equalities rather than replacing terms, and Datalog, union-find and lattices live in one engine. Szalinski used this to decompile flat programs into structured ones. This is the knowledge store.
- **Assumption-based truth maintenance** [9]: facts labelled with the assumptions they rest on, and nogoods. This is how claims, retractions and blast radii work.
- **Patch theory** [10]: deletions as permanent marks rather than erasure; the version-control view of forward edits.
- **Binary type inference** [11], [12]: constraint-based recovery of types from machine code, a source of more saturation rules.

Part B — A realistic system

13 Architecture



Arrows: premises flow into the store; raises go to the checker and statuses come back; every store entry is logged and indexed; views flow to agents; agents submit to the orchestrator, which merges into the store at barriers; retrieved context flows from the memory index to agents.

- **Lifter + premise extractor.** Produces the initial AsmWebIR and the premises: kinds, accesses, live-ins, dataflow, CFG, dependences. A production lifter (e.g. from Ghidra's p-code or angr's VEX) is wrapped; only the translation to AsmWebIR is new.
- **Knowledge store.** An egglog-style engine: e-graph for terms and types, union-find for webs, lattices for kinds, layouts and bounds, and Datalog-style rules for saturation. Every entry has a label, i.e. the set of claims it depends on.
- **Edit log.** An append-only record of every entry and submission, with its consequences. It is the source of truth for replay and for the memory index.
- **View server.** Extracts per-function views, and updates them incrementally with Pantograph when facts arrive.
- **Checker.** Discharges raises: SMT or symbolic execution first, differential testing on timeout.
- **Memory index (RAG).** Makes the log and the program's entities searchable by agents (§Memory).
- **Orchestrator.** Assigns functions to agents, holds barriers, and runs synchronisation (§Agents).

14 Memory: retrieval over edits and entities

The store answers exact questions ("what is the type of web `#d1`?"). Agents also need *fuzzy* questions: "have we seen a function like this?", "why was `val` made 32-bit?", "what did the other agent conclude about this struct?". A retrieval index answers these.

14.1 What is indexed

Document kind	Content	Embedded from
Edit record	a claim, retraction, submission or raise; its rationale text; its consequences (facts derived, view nodes changed, nogoods produced)	rationale + rendered before/after view snippet
Nogood record	the claims involved, the witness instructions, the eventual resolution	witness + resolution text

Document kind	Content	Embedded from
Entity card	one per function, web, struct, global, variable, or hole: current beliefs, status, open holes, obligations, callers/callees, a summary written by the agent that completed it	summary + signature + normalised assembly
Precedent	completed functions from earlier binaries or libraries (optional): normalised assembly, final view, claims	normalised assembly
External reference	library signatures, ABI notes, known compiler idioms	text

Normalised assembly means addresses, immediates and register names are canonicalised, so that structurally similar functions embed close together.

14.2 Rules for using retrieval safely

Retrieval is an **index, never a source of truth**. Four rules keep it from becoming a new source of hallucination:

1. **Pointers, not facts.** Every retrieved document carries the ids of the store entries it describes. Before it is shown to an agent, the document is re-rendered from the store at the current epoch, so stale text is never shown as current.
2. **Retrieval adds nothing to the store.** To use a precedent ("the similar function in `libfoo` had `struct Node`"), the agent must submit a claim. The claim gets a new label and is checked like any other. It records the precedent as its justification, for audit.
3. **Nogoods are retrieved first.** When an agent opens a function, every nogood touching that function's webs is put in context before anything else. This is the cheapest hallucination filter available.
4. **Every agent writes, one curator summarises.** Agents append rationale text to their edits. Entity summaries are written when a function completes, and regenerated when a claim they rely on is retracted.

14.3 Context assembly for an agent

When an agent opens function F, its prompt is built from:

1. the view of F, with marks, holes and obligations;
2. entity cards for F's **frontier webs**, i.e. the webs shared with other functions;
3. the nogoods touching those webs (rule 3 above);
4. the top-k precedents by normalised-assembly similarity, with their final views;
5. recent edit records by other agents on the same frontier webs.

The token budget is split in that order. Items 1–3 are exact and item 4 is fuzzy.

15 Agents: isolation, completion barriers, synchronisation

15.1 Work units and scheduling

The unit of work is a **function**. A strongly connected component of the call graph (mutual recursion) is one unit. The orchestrator schedules **bottom-up** over the call graph, so callees tend to be finished before their callers and signatures flow upward. Units that touch many frontier webs are prioritised, since their completion informs the most other units.

15.2 Local and global claims

Relative to its function F, each agent's claims fall into two classes:

- **Local claims** mention only webs whose members all lie in F: temporaries, non-escaping stack objects, local restructuring, raises inside F. They cannot affect any other function, so they go into the shared store **immediately**.
- **Global claims** mention a **frontier web**: F's own signature, the types of pointers it shares, the names of shared structs. These are **staged** in the agent's private **overlay**. The agent sees them in its own view, and nobody else does.

The agent's view is the extraction of (shared store $\cup$ its overlay). Other agents never see a half-finished global decision.

15.3 The protocol

Phase	What happens
Work	Agents edit drafts, submit claims. Local claims go straight to the store, global claims to the agent's overlay. Agents may read anything and retrieve from memory.
Complete	An agent declares F complete when: no residual is left un-raised, every hole is filled or explicitly left opaque, its view has no active marks, and its raises have been checked. The orchestrator verifies this.
Pause	The orchestrator raises a barrier. Affected agents (below) finish their current step and stop submitting. Drafts are kept, since they are private.
Propagate	The completed agent's overlay is merged into the store and saturation runs. New nogoods are computed with their labels, and the memory index is updated with the edits, their consequences, and F's entity card.
Sync	Every paused agent's view is recomputed against the new store plus its own overlay, and it is shown the view diff (via Pantograph). Staged claims that are now *derived* are marked redundant. Staged claims now in a nogood are flagged to their owner.
Resume	The barrier lifts and work continues.

Who pauses. A stop-the-world barrier is simplest. A **soft barrier** pauses only agents whose function has a frontier web mentioned by the overlay being merged. The others cannot observe the change, so they keep working.

Why agents do not interfere. (i) Local claims touch disjoint webs, so they cannot change another agent's view. (ii) Global claims are invisible until a barrier, so no agent's view changes under it mid-step. (iii) Merging is set union, so the order of merges does not matter. (iv) Every conflict is a nogood whose label names the claims, and therefore the agents, involved.

15.4 Conflict policy

When a sync produces a nogood between a newly merged claim and an agent's staged claim:

1. If one side is derived from premises alone (empty label), the other side is wrong. Its owner is told to retract.
2. Otherwise the **completed** function's claim wins by default, because it passed the completion check. The staged side is flagged, not deleted.
3. Disagreements about names (not a contradiction, but two names for one web) go to a **reviewer** agent. The reviewer sees both functions' entity cards and the retrieval context, and decides by submitting a retraction for one side.
4. A retraction of a merged claim re-opens every **completed** function whose view depended on it. The blast radius is computed from labels, and those functions are re-queued.

15.5 Worked timeline on the running example

t	Agent A: `list_sum`	Agent B: `list_push`	Agent C: `h_data`	Store
1	stages `Node`, `?T1 = Node` (frontier `#d1`)	stages `ListItem {0: next; 8: val: i32}` for `#d1`	local: names temporaries	locals of C
2	local: restructures loop (derived)	waits for `malloc` card (retrieved)	stages `len: u64` on `#c4`	adds A's loop eq.
3	**completes** → barrier on `#d1`	paused (frontier `#d1`)	keeps working (soft barrier)	
4	merged: `Node`, names	sync: its `ListItem` names web `#d1` too → naming conflict → reviewer		adds `Node`; conflict
5	takes next unit (`main`)	retracts `ListItem`; view now shows `Node` fields	**completes** → barrier on `#c4`	adds C's claims
6	`main` view shows `h: ptr Node`	completes	takes `h_ping` (already shows `len`)	…

16 The agent's tool interface

Tool	Effect
`view(f)`	current view of f with holes, marks, obligations, and frontier webs
`why(node)`	the derivation of a view node: its label (claims) and its premises (instructions)
`draft.open(f)` / `draft.edit(op)`	scratch copy; Pantograph operations: insert or delete context, cut/paste span, edit annotation
`draft.submit(rationale)`	diff → claims and proposals; reply lists derived / raised / nogoods / staged
`claim(fact, rationale)`	direct claim without a draft (types, names, declarations)
`retract(id, reason)`	add a retraction; the reply lists the blast radius
`blast(id)`	dry run: which view nodes and functions depend on a claim
`memory.search(query, kinds)`	retrieval over edits, nogoods, entity cards, precedents
`complete(f)`	request completion; the orchestrator checks the criteria and runs the barrier

17 Engineering notes

- **Store.** egglog, or a Datalog engine plus an e-graph library, extended with labels. Labels can be kept as bitsets over claim ids. If minimal-label (ATMS) maintenance is too expensive, fall back to a single justification per fact, at the cost of looser blast radii.
- **Log and index.** A relational log (e.g. Postgres) with a vector extension, or a separate vector database, fed by change-data-capture from the log. Hybrid retrieval: vector similarity plus keyword/identifier match. Addresses and web ids are exact keys.
- **Archiving.** Entries whose labels are all retracted leave the saturation set but stay queryable.
- **Checker.** Bit-precise SMT over the lifted region (e.g. Bitwuzla or Z3), with a time budget. On timeout, run differential testing on the region under the kinds of its live-ins.

- **Output.** C is emitted from the final views. Opaque holes become `struct unk_N { char _[size]; }` with the known fields filled in. Each construct can carry a confidence comment (certain, claimed, raised: tested).

18 Evaluation plan (sketch)

- **Accuracy.** Types, signatures and struct layouts against debug information of binaries built with `-g` and then stripped. Names judged against the originals by an embedding similarity.
- **Faithfulness.** Recompile the output and compare behaviour by differential testing; count raises by status.
- **Hallucination.** Rate of claims that end in nogoods; rate of retractions after sync; repeats of known nogoods (should be 0 with retrieval).
- **Cost.** Tokens per function, barriers per binary, agent idle time under soft vs hard barriers.
- **Baselines.** Hex-Rays and Ghidra; LLM-only decompilers; LLM refinement of decompiler output.

Part C — The formal rules

This part lists every rule. Each rule is followed by a **Does:** line that says what it accomplishes in plain words, so the rules can be checked selectively. Rules from earlier drafts that are no longer used are listed in Section 30.

19 Syntax

19.1 Types, kinds, terms

Web ids	u	::=	$c \mid d \mid w$ code, data, value webs
Provenance	ρ	::=	$[a, a') \mid \rho \cup \rho \mid \varepsilon$
Classes	k	::=	$\mathsf{int} \mid \mathsf{flt} \mid \mathsf{ptr} \mid \mathsf{agg} \mid \top$
Kinds	κ	::=	$[n, m, k]$ size (possibly $\geq n$), alignment, class
Types	τ, σ	::=	$\mathsf{i}N \mid \mathsf{u}N \mid \mathsf{f}N \mid \mathsf{void} \mid ?\alpha \mid \mathsf{ptr}^d\tau \mid S^d \mid \tau[\nu]^d \mid \langle\overline{\tau}\rangle \xrightarrow{c} \tau$
Lengths	ν	::=	$n \mid ?n$
Structs	S	::=	$\mathsf{struct}\, s \left\{\overline{o : f : \tau}\right\} \mathsf{size}\ n$
Expressions	e	::=	$x^w \mid n \mid e \oplus e \mid (\tau)e \mid e +_b e \mid \mathsf{load}_\kappa(e) \mid e \xrightarrow{d} f \mid e[e]^d \mid \mathsf{call}^c e(\overline{e}) \mid \square^\rho_\tau$
Draft-only		|	$\{e\}^!_{\tau/\tau} \mid \{e\}^\uparrow_{\Delta\vdash\delta} \mid \{e\}^\downarrow_{\Delta\vdash\delta} \mid /\!/\, x$
Statements	s	::=	$\mathsf{let}\ x^w : \tau = e \mid \ell := e \mid \mathsf{if}\ e\, s\, s \mid \mathsf{while}\ e\, s \mid \mathsf{return}\ e \mid \overline{s} \mid \square^\rho$
Functions	F	::=	$\mathsf{fn}^c f@a(\overline{x^w : \tau}) : \tau = s$
Diffs	δ	::=	$l\,\overline{\delta} \mid {}^+\langle c\langle\delta\rangle\rangle \mid {}^-\langle c\langle\delta\rangle\rangle \mid \tau/\tau'$

19.2 Knowledge

Claims	a	$\in$	claim identifiers; each claim is owned by one agent
Labels	L	::=	finite sets of claims ($\emptyset$ = certain)
Facts	φ	::=	$\mathsf{bin}(\rho, h) \mid n \doteq n' \mid t : \tau \mid \tau :> \lfloor o : \kappa \rfloor \mid \tau \Downarrow o : \sigma \mid \mathsf{arity}(c) \geq i$
		|	$?n \geq k \mid \mathsf{decl}(S) \mid \mathrm{name}(u, s) \mid \mathrm{dead}(n) \mid \bot$
Store	$\mathfrak{K}$	$\in$	sets of: $\varphi\,[L]$, $a : \varphi$, $\mathsf{retract}(a, r)$, $\mathsf{nogood}(L)$, $\mathsf{obl}(a, \pi)$
Beliefs	$\mathcal{E}(\mathfrak{K})$	=	$\{a \mid (a : \varphi) \in \mathfrak{K}, \mathsf{retract}(a, \cdot) \notin \mathfrak{K}\}$
Active	$\mathfrak{K} \vdash_{\mathcal{E}} \varphi$	$\iff$	$\exists L \subseteq \mathcal{E}.\ \varphi\,[L] \in \mathfrak{K}^*$
Agent state	A_i	=	$\langle F_i, O_i, D_i\rangle$: assigned function, overlay (staged claims), draft

20 Kinds and layouts

$$\frac{}{\Sigma \vdash \mathsf{i}N :: \left[\tfrac{N}{8}, \tfrac{N}{8}, \mathsf{int}\right]}\ \text{K-Int}$$

Does: gives an integer type the kind the machine sees: $\frac{N}{8}$ bytes, aligned to its size.

$$\frac{}{\Sigma \vdash \mathsf{ptr}^d\tau :: [8, 8, \mathsf{ptr}]}\ \text{K-Ptr}$$

Does: all pointers are 8-byte, 8-aligned pointer-class values, whatever they point to.

$$\frac{?\alpha :: \kappa \in \Sigma}{\Sigma \vdash ?\alpha :: \kappa}\ \text{K-Hole}$$

Does: an unknown type still has the kind recorded for it from the instructions.

$$\frac{\forall i.\, \Sigma \vdash \tau_i :: [n_i, m_i, k_i] \qquad o_i \equiv 0 \pmod{m_i} \qquad o_i + n_i \le o_{i+1}}{\Sigma \vdash S^d :: [n, \max m_i, \mathsf{agg}]}\ \text{K-Struct}$$

Does: a struct is well formed if its fields are aligned and do not overlap; its alignment is the largest field alignment.

$$\frac{\Sigma \vdash \tau :: [n, m, k]}{\Sigma \vdash \tau[\nu]^d :: [\nu \cdot n, m, \mathsf{agg}]}\ \text{K-Arr}$$

Does: an array is ν copies of its element, with the element's alignment.

$$\frac{}{\Sigma \vdash \tau \Downarrow 0 : \tau}\ \text{L-Zero}$$

Does: at offset 0 of any value there is the value itself (so `*p` is `load(p + 0)`).

$$\frac{S(o) = f : \sigma}{\Sigma \vdash S^d \Downarrow o : \sigma}\ \text{L-Field}$$

Does: at a field's offset lives that field's type.

$$\frac{S(o_1) = f : \sigma_1 \qquad \Sigma \vdash \sigma_1 \Downarrow o_2 : \sigma}{\Sigma \vdash S^d \Downarrow o_1 + o_2 : \sigma}\ \text{L-Nest}$$

Does: offsets reach into nested structs.

$$\frac{\Sigma \vdash \tau :: [n, m, k] \qquad \Sigma \vdash \tau \Downarrow (o \bmod n) : \sigma}{\Sigma \vdash \tau[\nu]^d \Downarrow o : \sigma}\ \text{L-Elem}$$

Does: offsets inside arrays reduce to offsets inside one element.

$$\frac{?\alpha :> \lfloor \ldots, o : \sigma, \ldots \rfloor \in \Sigma}{\Sigma \vdash ?\alpha \Downarrow o : \sigma}\ \text{L-Open}$$

Does: an unknown struct answers lookups at the offsets where evidence has been recorded.

21 Typing of terms

These rules type views and drafts. Σ is the hole store as seen from $\mathfrak{K}$ (kinds, open layouts, solved holes).

$$\frac{x^w : \tau \in \Gamma}{\Sigma; \Gamma \vdash x^w : \tau}\ \text{T-Var}$$

Does: a variable has its declared type.

$$\frac{\Sigma; \Gamma \vdash e : \mathsf{ptr}^d \tau \qquad \Sigma \vdash \tau \Downarrow o : \sigma \qquad \Sigma \vdash \sigma :: \kappa}{\Sigma; \Gamma \vdash \mathsf{load}_\kappa(e +_b o) : \sigma}\ \text{T-Load}$$

Does: a raw memory read is well typed when the pointee has, at that offset, something of the width and class read. This is the central decompilation rule.

$$\frac{\Sigma; \Gamma \vdash e : \mathsf{ptr}^d S^d \qquad S(o) = f : \sigma}{\Sigma; \Gamma \vdash e \xrightarrow{d} f : \sigma}\ \text{T-Field}$$

Does: a field access has the field's type.

$$\frac{\Sigma; \Gamma \vdash e_1 : \mathsf{ptr}^d\big(\tau[\nu]^d\big) \qquad \Sigma; \Gamma \vdash e_2 : \mathsf{i}N}{\Sigma; \Gamma \vdash e_1[e_2]^d : \tau}\ \text{T-Index}$$

Does: indexing a pointer to an array yields an element.

$$\frac{\Sigma;\Gamma \vdash e : \langle \overline{\tau} \rangle \xrightarrow{c} \tau \qquad \Sigma;\Gamma \vdash \overline{e} : \overline{\tau}}{\Sigma;\Gamma \vdash \mathsf{call}^{c} e(\overline{e}) : \tau}\ \text{T-Call}$$

Does: a call is well typed when its arguments match the parameter types of its code web.

$$\frac{\Sigma;\Gamma \vdash e : \sigma \qquad \Sigma \vdash \sigma :: \kappa \qquad \Sigma \vdash \tau :: \kappa}{\Sigma;\Gamma \vdash (\tau) e : \tau}\ \text{T-Cast}$$

Does: casts may only reinterpret between types of the same machine kind; anything else is real computation and must come from the binary.

$$\frac{\Sigma \vdash \tau :: \kappa \qquad B \vdash \rho \vDash \kappa}{\Sigma;\Gamma \vdash \square^{\rho}_{\tau} : \tau}\ \text{T-Resid}$$

Does: un-lifted assembly may be given any type whose kind matches what the code actually produces.

$$\frac{\Sigma;\Gamma, \overline{x^{w} : \tau} \vdash s\ \mathsf{ok} \qquad B \vdash a \vDash \mathsf{abi}(\overline{\tau} \to \tau')}{\Sigma;\Gamma \vdash \left(\mathsf{fn}^{c} f@a(\overline{x^{w} : \tau}) : \tau' = s\right) : \langle \overline{\tau} \rangle \xrightarrow{c} \tau'}\ \text{T-Fn}$$

Does: a function's signature must match the calling convention at its address: parameter count and kinds fit the live-in registers and stack slots.

$$\frac{\Sigma;\Gamma \vdash e : \tau_1}{\Sigma;\Gamma \vdash \{e\}^{!}_{\tau_1/\tau_2} : \tau_2}\ \text{T-Err}$$

Does: (drafts only) an error boundary lets a term of the wrong type sit in a context expecting another type, visibly.

$$\frac{\Sigma;\Delta.2 \vdash e : \delta.2}{\Sigma;\Delta.1 \vdash \{e\}^{\uparrow}_{\Delta \vdash \delta} : \delta.1}\ \text{T-Up}$$

Does: (drafts and view updates) an up-boundary marks a term whose type changed by δ and whose context has not caught up yet.

$$\frac{\Sigma;\Delta.1 \vdash e : \delta.1}{\Sigma;\Delta.2 \vdash \{e\}^{\downarrow}_{\Delta \vdash \delta} : \delta.2}\ \text{T-Down}$$

Does: a down-boundary marks a term whose expected type changed by δ and which has not caught up yet.

22 Webs and locality

$$\frac{t : \mathsf{ptr}^{d_1} \tau_1 \qquad t : \mathsf{ptr}^{d_2} \tau_2}{d_1 \doteq d_2 \quad \tau_1 \doteq \tau_2}\ \text{W-Unify}$$

Does: if one value is seen with two pointer types, the two types and their webs are the same (the same for every type former). This is how webs are inferred.

$$\frac{\mathsf{members}(u) \subseteq F}{u \in \mathsf{Loc}(F)}\ \text{W-Local}$$

Does: a web whose members are all in one function is local to it.

$$\frac{\mathsf{members}(u) \cap F \neq \emptyset \qquad \mathsf{members}(u) \not\subseteq F}{u \in \mathsf{Fr}(F)}\ \text{W-Frontier}$$

Does: a web that is partly in F and partly elsewhere is on F's frontier.

$$\frac{\mathsf{fp}(\varphi) = \{u \mid u \text{ occurs in } \varphi \text{ up to } \doteq\}}{\varphi \text{ is local to } F \iff \mathsf{fp}(\varphi) \subseteq \mathsf{Loc}(F)}\ \text{W-Footprint}$$

Does: classifies a claim: it is local if every web it mentions is local to the agent's function; otherwise it is global and must be staged.

23 The knowledge store: saturation

Saturation $\mathfrak{K}^*$ is the least fixpoint of the following rules. Each adds a fact whose label is the union of its premises' labels.

$$\frac{\mathsf{bin}(\rho, \mathsf{access}(e, o, \kappa))\,[\emptyset] \qquad e : \mathsf{ptr}^{d}\tau\,[L]}{\tau :> \lfloor o : \kappa \rfloor\,[L]}\ \text{S-Access}$$

Does: each memory access in the binary adds "there is something of this kind at this offset" to the pointee type. This builds open layouts.

$$\frac{\mathsf{bin}(\rho, \mathsf{livein}(f, \mathsf{reg}_i))\,[\emptyset] \qquad f \in c\,[L]}{\mathsf{arity}(c) \geq i\,[L]}\ \text{S-ABI}$$

Does: if any function in a code web reads the i-th argument register, every function in the web has at least i parameters.

$$\frac{\mathsf{arity}(c) \geq i\,[L] \qquad \mathsf{call}^{c}_{\rho} e(\overline{e}) \in \mathfrak{K} \qquad \mathsf{bin}(\rho, \mathsf{reg}_i = e_{\text{new}})\,[\emptyset]}{\mathsf{arg}_i(\rho) \doteq e_{\text{new}}\,[L]}\ \text{S-Arg}$$

Does: at every call in the web, the i-th argument is whatever the binary puts in the i-th argument register there (the hint oracle).

$$\frac{\mathsf{bin}(\rho, \mathsf{index}(e, i \leq k))\,[\emptyset] \qquad e : \mathsf{ptr}^{d}\big(\tau[?n]^{d}\big)\,[L]}{?n \geq k + 1\,[L]}\ \text{S-Bound}$$

Does: an observed index bound gives a lower bound on an array's length.

$$\frac{t : \tau_1\,[L_1] \qquad t : \tau_2\,[L_2]}{\tau_1 \doteq \tau_2\,[L_1 \cup L_2]}\ \text{S-Unify}$$

Does: a term has one type: two typings are equated (and, by W-Unify, their webs merged).

$$\frac{\tau \doteq S^{d}\,[L_1] \qquad S(o) = f : \sigma\,[L_2] \qquad n \in [\mathsf{load}_{\kappa}(e +_b o)] \qquad e : \mathsf{ptr}^{d}\tau\,[L_3]}{n \doteq \left(e \xrightarrow{d} f\right)\,[L_1 \cup L_2 \cup L_3]}\ \text{S-Field}$$

Does: once a pointee is believed to be a struct, raw loads at field offsets become equal to field accesses, in every function of the web. This is "structify", and it adds an alternative rather than rewriting.

$$\frac{\tau \doteq \sigma[\nu]^{d}\,[L] \qquad n \in [\mathsf{load}_{\kappa}(e +_b (|\sigma| \cdot i))]}{n \doteq e[i]^{d}\,[L]}\ \text{S-Index}$$

Does: strided loads through an array pointer become equal to indexing ("arrayify").

$$\frac{n \in [t_1] \qquad t_1 \rightsquigarrow t_2 \in \mathcal{R}_{\text{struct}} \qquad \mathsf{side}(t_1 \rightsquigarrow t_2)\,[L]}{n \doteq t_2\,[L]}\ \text{S-Rewrite}$$

Does: the library of sound structuring rewrites (loop rotation, branch inversion, `goto` removal, `for` formation) adds equal alternatives; any side condition's label is inherited.

$$\frac{\tau :> \lfloor o : \kappa \rfloor\,[L_1] \qquad \tau \Downarrow o : \sigma\,[L_2] \qquad \sigma :: \kappa' \qquad \kappa \not\leq \kappa'}{\bot\,[L_1 \cup L_2]}\ \text{S-Clash}$$

Does: if what a type is believed to contain at an offset has a different kind from what the binary accesses there, that is a contradiction.

$$\frac{\mathsf{arity}(c) \geq i\,[L_1] \qquad c \doteq \langle \tau_1 ... \tau_j \rangle \to \tau\,[L_2] \qquad j < i}{\bot\,[L_1 \cup L_2]}\ \text{S-Clash-Arity}$$

Does: a signature with fewer parameters than the binary requires is a contradiction.

$$\frac{?n \geq k\,[L_1] \qquad ?n \doteq m\,[L_2] \qquad m < k}{\bot\,[L_1 \cup L_2]}\ \text{S-Clash-Bound}$$

Does: an array length smaller than an observed index is a contradiction.

$$\frac{\mathsf{obl}(a, \mathsf{refuted}(v)) \in \mathfrak{K}}{\bot\,[\{a\}]}\ \text{S-Refute}$$

Does: a raise whose check found a counter-example v is a contradiction.

$$\frac{\bot\,[L]}{\mathsf{nogood}(L)}\ \text{S-Nogood}$$

Does: records every contradiction as a nogood: the set of claims that cannot all hold.

$$\frac{\mathsf{name}(u, s_1)\,[L_1] \qquad \mathsf{name}(u, s_2)\,[L_2] \qquad s_1 \neq s_2}{\mathsf{conflict}(L_1, L_2)}\ \text{S-Name-Conflict}$$

Does: two different names for the same web are a (non-semantic) conflict, shown as a mark and sent to review; not a nogood.

24 Views

$$\frac{V = \mathsf{extract}_F(\mathfrak{K}^*, \mathcal{E}, \mathsf{cost}, V^{\text{prev}}) \qquad \forall n \in V.\, \exists L \subseteq \mathcal{E}.\, n \doteq \mathsf{chosen}(n)\,[L] \in \mathfrak{K}^*}{V \text{ is the view of } F}\ \text{V-Extract}$$

Does: a view picks, for each e-class in F, one member that is active under current beliefs, minimising a cost (structured < raw, certain < claimed, close to the previous view).

$$\frac{n \text{ is lifted from } \rho}{n \doteq t_\rho\,[\emptyset] \in \mathfrak{K}^*}\ \text{V-Raw}$$

Does: the raw lifted term is always an active member, so extraction always succeeds and faithfulness has a fallback.

$$\frac{\mathsf{nogood}(L) \in \mathfrak{K}^* \qquad L \subseteq \mathcal{E} \qquad n \in V \qquad \mathsf{label}(\mathsf{chosen}(n)) \cap L \neq \emptyset}{n \text{ is marked with } L \text{ and its witness}}\ \text{V-Mark}$$

Does: an active contradiction is displayed on the smallest view nodes that depend on it, with the instructions that witness it.

$$\frac{V' = \mathsf{extract}_F(\mathfrak{K}'^*, \ldots) \qquad \mathsf{diff}(V, V') = \overline{\delta}}{V\{\cdot\}^{\uparrow}_{\delta} \xrightarrow{*} V'}\ \text{V-Update}$$

Does: when the store grows, the view is updated by seeding Pantograph boundaries where members changed and propagating them (P-rules), rather than re-rendering.

25 Drafts: Pantograph editing

Drafts use Pantograph's rules as published [2]. We list the instances that matter.

25.1 Propagation

$$\frac{}{\mathsf{call}^c\{e\}^{\uparrow}_{\overline{\delta} \xrightarrow{c} \delta}(\overline{e}) \longrightarrow \left\{\mathsf{call}^c e\left(\overline{\{e\}^{\downarrow}_{\delta}}\right)\right\}^{\uparrow}_{\delta}}\ \text{P-Call-Fn}$$

Does: if a function's type changes, its arguments are told their expected types changed, and the call's result type changes.

$$\frac{}{\mathsf{call}^c e\left(\ldots, \{e_i\}^{\uparrow}_{\delta}, \ldots\right) \longrightarrow \mathsf{call}^c\{e\}^{\downarrow}_{\langle \ldots, \delta, \ldots\rangle \xrightarrow{c} \mathrm{id}}(\ldots, e_i, \ldots)}\ \text{P-Call-Arg}$$

Does: if an argument's type changes, the function being called is told its parameter type changed.

$$\frac{}{\{x\}^{\downarrow}_{(\Delta, x:\delta) \vdash \mathrm{id}} \longrightarrow \{x\}^{\uparrow}_{\delta}}\ \text{P-Var}$$

Does: if a variable's declared type changed, each use of it reports the change upward.

$$\frac{\{e\}^{\uparrow}_{\mathsf{ptr}^{d}(S/S')} \xrightarrow{d} f \longrightarrow e \xrightarrow{d} f' \qquad S(o) = f : \sigma}{S'(o) = f' : \sigma}\ \textsc{P-Field}$$

Does: if a pointer's struct type changes but the new struct has a field of the same type at the same offset, the access is re-pointed to that field.

25.2 Alteration (where a boundary stops)

$$\frac{\mathcal{H}(\rho, i, \tau) = e_{\text{new}}}{\mathsf{call}^{c}_{\rho}\{e\}^{\uparrow}_{\langle \mathrm{id}, +_i \tau \rangle \xrightarrow{c} \mathrm{id}}(\overline{e}) \longrightarrow \mathsf{call}^{c}_{\rho} e(\overline{e}[i \mapsto e_{\text{new}}])}\ \textsc{A-AddArg}$$

Does: a call to a function that gained a parameter gets the argument the binary actually passes (S-Arg), or a hole if unknown.

$$\frac{y \text{ fresh}}{\{\mathsf{fn}^{c} f(\overline{x : \tau}) = s\}^{\downarrow}_{\langle \mathrm{id}, +_i \tau \rangle \xrightarrow{c} \mathrm{id}} \longrightarrow \mathsf{fn}^{c} f(\overline{x : \tau}[i \mapsto y : \tau]) = s[y/\mathsf{reg}_i]}\ \textsc{A-AddParam}$$

Does: a function that gains a parameter binds it, and reads of the matching register inside residual code now refer to it.

$$\frac{}{\mathsf{call}^{c}\{e\}^{\uparrow}_{\langle \mathrm{id}, -_i \tau \rangle \xrightarrow{c} \mathrm{id}}(\overline{e}) \longrightarrow \mathsf{call}^{c} e(\overline{e} \setminus e_i)\ /\!\!/\ e_i}\ \textsc{A-DropArg}$$

Does: a call to a function that lost a parameter keeps the dropped argument as a visible comment instead of deleting it.

$$\frac{\text{no other rule applies}}{\{t\}^{\downarrow}_{\tau_1/\tau_2} \longrightarrow \{t\}^{!}_{\tau_1/\tau_2}}\ \textsc{A-Error}$$

Does: when a type change cannot be absorbed automatically, an explicit error boundary is left for the agent.

25.3 Healing

$$\frac{}{\{e\}^{!}_{\tau/\tau} \longrightarrow e}\ \textsc{H-Err}$$

Does: an error boundary whose two types have become equal disappears.

$$\frac{x : \tau \in \Gamma}{\Gamma \vdash /\!\!/\ x^{\tau} \longrightarrow x}\ \textsc{H-Var}$$

Does: a free-variable mark disappears once its binder is back in scope.

$$\frac{e : \langle \overline{\tau}, \tau' \rangle \xrightarrow{c} \sigma}{\mathsf{call}^{c} e(\overline{e})\ /\!\!/\ e_i \longrightarrow \mathsf{call}^{c} e(\overline{e}, e_i)}\ \textsc{H-Arg}$$

Does: a commented-out argument is restored if the function takes that parameter again.

25.4 Statement spans

$$\frac{\overline{s} \text{ binds } \overline{x : \tau}}{Z[\overline{s}] \longrightarrow Z[\,]^{\downarrow}_{-\langle \overline{x:\tau} \rangle \vdash \mathrm{id}} \quad \text{clipboard} := \overline{s}}\ \textsc{Seg-Cut}$$

Does: cutting a span of statements removes its bindings from what follows, so later uses become free-variable marks. This is how loop and branch restructuring is expressed.

$$\frac{\overline{s} \text{ binds } \overline{x : \tau}}{Z[\,] \longrightarrow Z[\overline{s}]^{\downarrow}_{+\langle \overline{x:\tau} \rangle \vdash \mathrm{id}}}\ \textsc{Seg-Paste}$$

Does: pasting a span re-introduces its bindings, and H-Var heals uses that are now in scope.

26 Edits and submission

$$\frac{a \text{ fresh, owned by agent } i}{\mathfrak{K} \vdash \mathsf{claim}\ \varphi \Rightarrow \mathfrak{K} \cup \{a : \varphi\}}\ \textsc{E-Claim}$$

Does: records an agent's claim as an assumption with a new label.

$$\frac{(a : \varphi) \in \mathfrak{K}}{\mathfrak{K} \vdash \mathsf{retract}\ a\ \text{because}\ r \Rightarrow \mathfrak{K} \cup \{\mathsf{retract}(a, r)\}}\ \text{E-RETRACT}$$

Does: records that a claim is withdrawn, and why; nothing is removed.

$$\frac{n \doteq n'\ [L] \in \mathfrak{K}^*}{\mathfrak{K} \vdash \mathsf{propose}\ n \doteq n' \Rightarrow \mathfrak{K}}\ \text{E-DERIVED}$$

Does: a proposed rewrite that the engine can already derive adds nothing; the agent is told it is derived, and under which claims.

$$\frac{n \doteq n' \notin \mathfrak{K}^* \qquad \mathsf{check}(n, n') = \pi \qquad a\ \text{fresh}}{\mathfrak{K} \vdash \mathsf{propose}\ n \doteq n' \Rightarrow \mathfrak{K} \cup \{a : n \doteq n', \mathsf{obl}(a, \pi)\}}\ \text{E-RAISE}$$

Does: a proposed replacement that cannot be derived becomes a claim with a checking status (proved, tested, or refuted).

$$\frac{a\ \text{fresh} \qquad \mathsf{evidence}(n) \in \{\mathsf{premise}, \mathsf{proved}\}}{\mathfrak{K} \vdash \mathsf{claim}\ \mathrm{dead}(n) \Rightarrow \mathfrak{K} \cup \{a : \mathrm{dead}(n)\}}\ \text{E-DEAD}$$

Does: claims that *hide* something (unused parameter, unreachable code) need premise-level or proved evidence.

$$\frac{\mathsf{diff}(V_F, D) = \overline{c} \qquad \Delta = \bigcup_i \mathsf{tr}(c_i)}{\mathfrak{K} \vdash \mathsf{submit}\ D \Rightarrow \mathfrak{K} \cup \Delta}\ \text{E-SUBMIT}$$

Does: submitting a draft translates each Pantograph change into claims and proposals (table below); the draft itself never enters the store.

Change in the draft	**tr(c)**
type diff δ on a former of web u	claim $\tau_u \doteq \delta.2$
subterm at e-class n replaced by t'	$\mathsf{propose}\ n \doteq t'$ (E-Derived or E-Raise)
parameter inserted	claim $\mathsf{arity}(c) \geq i$ and its type
code deleted, nothing in its place	claim $\mathrm{dead}(n)$ (E-Dead)
rename	claim $\mathrm{name}(u, s)$

27 Multi-agent protocol

The shared store is $\mathfrak{K}$. Each agent i has state $A_i = \langle F_i, O_i, D_i \rangle$.

$$\frac{V_i = \mathsf{extract}_{F_i}((\mathfrak{K} \cup O_i)^*, \mathcal{E}(\mathfrak{K} \cup O_i))}{\text{agent } i \text{ sees } V_i}\ \text{M-VIEW}$$

Does: an agent sees the shared store plus its own staged claims, and nobody else's staged claims.

$$\frac{\varphi\ \text{is local to}\ F_i \qquad \mathfrak{K} \vdash \mathsf{claim}\ \varphi \Rightarrow \mathfrak{K}'}{\langle \mathfrak{K}, O_i \rangle \xrightarrow[i]{\mathsf{claim}\ \varphi} \langle \mathfrak{K}', O_i \rangle}\ \text{M-LOCAL}$$

Does: a claim that only mentions webs local to the agent's function goes straight into the shared store: it cannot change anyone else's view.

$$\frac{\varphi\ \text{is not local to}\ F_i \qquad O_i \vdash \mathsf{claim}\ \varphi \Rightarrow O_i'}{\langle \mathfrak{K}, O_i \rangle \xrightarrow[i]{\mathsf{claim}\ \varphi} \langle \mathfrak{K}, O_i' \rangle}\ \text{M-STAGE}$$

Does: a claim that mentions a frontier web is kept in the agent's overlay until its function completes.

$$\frac{\mathsf{residuals}(V_i) \subseteq \mathsf{raised} \qquad \mathsf{holes}(V_i) \subseteq \mathsf{filled} \cup \mathsf{opaque} \qquad \mathsf{marks}(V_i) = \emptyset}{\mathsf{ready}(i)}\ \text{M-COMPLETE}$$

Does: a function is complete when all residual code is raised, every hole is filled or explicitly left opaque, and its view shows no contradictions.

$$\frac{\mathsf{ready}(i) \qquad P = \{j \mid \mathsf{Fr}(F_j) \cap \mathsf{fp}(O_i) \neq \emptyset\}}{\text{pause } P}\ \text{M-Barrier}$$

Does: when agent i completes, only the agents whose frontier shares a web with i's staged claims are paused (soft barrier).

$$\frac{\text{pause } P \qquad \mathfrak{K}' = \mathfrak{K} \cup O_i \qquad \mathsf{index}(\mathfrak{K}' \setminus \mathfrak{K})}{\langle \mathfrak{K}, O_i \rangle \Longrightarrow \langle \mathfrak{K}', \emptyset \rangle}\ \text{M-Merge}$$

Does: the completed agent's staged claims are merged into the shared store (set union, so order-independent), saturation runs, and the memory index is updated.

$$\frac{j \in P \qquad V_j' = \mathsf{extract}_{F_j}\big((\mathfrak{K}' \cup O_j)^*, \ldots\big) \qquad N_j = \big\{\mathsf{nogood}(L) \in (\mathfrak{K}' \cup O_j)^* \mid L \cap O_j \neq \emptyset\big\} \setminus \mathsf{old}}{\text{agent } j \text{ receives } \mathsf{diff}(V_j, V_j'), N_j}\ \text{M-Sync}$$

Does: each paused agent gets its recomputed view (as a Pantograph diff) and the new contradictions that involve its own staged claims.

$$\frac{(a : \varphi) \in O_j \qquad \varphi\,[L] \in \mathfrak{K}'^* \qquad L \subseteq \mathcal{E}(\mathfrak{K}')}{O_j \Rightarrow O_j \setminus \{a : \varphi\} \quad \text{(noted as confirmed)}}\ \text{M-Subsume}$$

Does: a staged claim that the shared store now derives on its own is dropped from the overlay as confirmed.

$$\frac{\forall j \in P.\, \text{agent } j \text{ acknowledged sync}}{\text{resume } P}\ \text{M-Resume}$$

Does: work continues once every paused agent has taken in its sync.

$$\frac{\mathsf{retract}(a) \in \mathfrak{K} \qquad a \in \mathsf{label}(\mathsf{chosen}(n)) \qquad n \in V_{F_k} \qquad F_k \text{ completed}}{\text{requeue } F_k}\ \text{M-Reopen}$$

Does: retracting a merged claim re-opens every completed function whose view depended on it.

$$\frac{\mathsf{nogood}(L) \qquad L = L_1 \cup L_2 \qquad L_1 \subseteq \mathsf{merged} \qquad L_2 \subseteq O_j}{\text{agent } j \text{ must retract from } L_2 \text{ or escalate}}\ \text{M-Policy}$$

Does: by default, in a conflict between a completed function's claims and a staged claim, the staged side yields (or asks the reviewer).

28 Memory (retrieval) rules

$$\frac{e \in \mathsf{log} \qquad \mathsf{doc}(e) = \langle \mathsf{text}(e), \mathsf{ids}(e), \mathsf{epoch} \rangle}{\mathsf{index} \ni (\mathsf{embed}(\mathsf{doc}(e)), \mathsf{doc}(e))}\ \text{R-Index}$$

Does: every log entry (claim, retraction, nogood, submission, completion) becomes a searchable document that points back to store ids.

$$\frac{\mathsf{hit} = \mathsf{doc}(e) \qquad \mathsf{shown} = \mathsf{render}(\mathsf{ids}(e), \mathfrak{K}_{\text{now}})}{\text{agent sees } \mathsf{shown}}\ \text{R-Render}$$

Does: a retrieved document is re-rendered from the current store before an agent sees it, so stale text is never shown as current.

$$\frac{\text{agent reads } \mathsf{hit}}{\mathfrak{K} \text{ unchanged}}\ \text{R-NoInject}$$

Does: reading from memory never changes the store; using a precedent requires submitting a new claim, which is checked like any other.

$$\frac{\text{agent opens } F \qquad N = \{\mathsf{nogood}(L) \mid \mathsf{fp}(L) \cap \mathsf{Fr}(F) \neq \emptyset\}}{\mathsf{context}(F) = N \cdot \mathsf{view}(F) \cdot \ldots}\ \text{R-Nogood-First}$$

Does: known contradictions about a function's shared webs are always put in the agent's context first.

29 Properties we expect (conjectures)

Forwardness. Every edit, merge and retraction only adds entries: $\mathfrak{K} \subseteq \mathfrak{K}'$ and $\mathfrak{K}^* \subseteq \mathfrak{K}'^*$.

Premise permanence. Facts with the empty label are active forever.

View soundness. Every view is well typed in the marked sense, and every inconsistency appears as a mark.

Anchoring. A view node whose label contains no raise is equal, by sound rewrites, to the lifted code, and so is faithful to B.

Retraction locality. A retraction changes a view only at nodes whose chosen member depends on the retracted claim.

Non-interference. Between barriers, agent j's view changes only through its own claims. Other agents' local claims cannot mention j's webs, and their global claims stay staged until a barrier.

Order independence. The store after any set of merges is the same, whatever the order.

Termination. Saturation terminates for a fixed rewrite library with bounded term depth. Pantograph propagation terminates (their Thm. 5). Each barrier merges one overlay.

30 Superseded rules

Old rule (part)	Replaced by	Why
E-Fill, E-Fill-Kind, E-Fill-Layout (I)	E-Claim + S-Clash	rejections became recorded nogoods
E-Structify, arrayify (I)	S-Field, S-Index	add equalities instead of rewriting
E-Split-Reject (I)	S-Unify + S-Nogood	a split is a claim that may clash with a unification chain
E-Retype, P-Web teleport (I)	E-Claim on a web + saturation + V-Update	global effect follows from labels, no teleport step needed
E-Accept (I)	E-Submit of a draft whose boundary was removed	boundary deletion now only happens in drafts
T-Split, P-Frontier, ledger (II)	M-View, M-Stage	overlay per agent instead of a diff ledger
Tx-Local, Tx-Global, Tx-Unhealed, Tx-Unanchored (II)	E-Submit, M-Complete, M-Merge	drafts are transactions; faithfulness via derivation
Tx-Commute (II)	Order independence (set union)	holds unconditionally
Anchor ($\underset{B}{\equiv}$) (II)	S-Rewrite + V-Raw	faithfulness is derivability from the lifted term

References

[1] B. Quiring, D. Van Horn, J. Reppy, and O. Shivers, "Webs and Flow-Directed Well-Typedness Preserving Program Transformations," *Proc. ACM Program. Lang. (PLDI)*, vol. 9, 2025, doi: 10.1145/3729280.

[2] J. Prinz, H. Blanchette, and L. Lampropoulos, "Pantograph: A Fluid and Typed Structure Editor," *Proc. ACM Program. Lang. (POPL)*, vol. 9, 2025, doi: 10.1145/3704864.

[3] E. Zhao, R. Maroof, A. Dukkipati, A. Blinn, Z. Pan, and C. Omar, "Total Type Error Localization and Recovery with Holes," *Proc. ACM Program. Lang. (POPL)*, vol. 8, 2024.

[4] M. D. Adams, E. Griffis, T. J. Porter, S. V. Satish, E. Zhao, and C. Omar, "Grove: A Bidirectionally Typed Collaborative Structure Editor Calculus," *Proc. ACM Program. Lang. (POPL)*, vol. 9, 2025.

[5] D. Moon, A. Blinn, and C. Omar, "Gradual Structure Editing with Obligations," in *IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, 2023.

[6] M. Willsey, C. Nandi, Y. R. Wang, O. Flatt, Z. Tatlock, and P. Panchekha, "egg: Fast and Extensible Equality Saturation," *Proc. ACM Program. Lang. (POPL)*, vol. 5, 2021.

[7] Y. Zhang *et al.*, "Better Together: Unifying Datalog and Equality Saturation," *Proc. ACM Program. Lang. (PLDI)*, vol. 7, 2023.

[8] C. Nandi *et al.*, "Synthesizing Structured CAD Models with Equality Saturation and Inverse Transformations," in *PLDI*, 2020.

[9] J. de Kleer, "An Assumption-Based TMS," *Artificial Intelligence*, vol. 28, 1986.

[10] S. Mimram and C. Di Giusto, "A Categorical Theory of Patches." [Online]. Available: https://arxiv.org/abs/1311.3903

[11] M. Noonan, A. Loginov, and D. Cok, "Polymorphic Type Inference for Machine Code," in *PLDI*, 2016.

[12] "BinSub: The Simple Essence of Polymorphic Type Inference for Machine Code." [Online]. Available: https://arxiv.org/abs/2409.01841